Package 'pons'

August 11, 2026

Title Interface to MS-Office for the DescToolsX ecosystem

Version 0.0.0.907

Description Provides Office routines used in DescToolsX.

Depends R (>= 4.2.0)

License GPL (>= 2)

Encoding UTF-8

Roxygen list(markdown = TRUE)

Imports RDCOMClient, bedrock, readxl, writexl, pharos

Suggests testthat (>= 3.0.0),
rstudioapi (>= 0.1), manipulate

Config/testthat/edition 3

LazyData true

Additional_repositories http://www.omegahat.net/R

URL https://andrisignorell.github.io/pons/,
https://github.com/AndriSignorell/pons/

BugReports https://github.com/AndriSignorell/pons/issues

Config/roxygen2/version 8.0.0

Contents

`bookmarkList` *List Word Bookmarks*

Description

Returns a data frame containing all bookmarks in the active Word document.

Usage

```
bookmarkList(wrd = NULL)
```

Arguments

`wrd`	A Word COM object. If `NULL`, the current active Word session is used.

Details

Bookmarks are returned in the order in which they appear in the document.

Value

A data frame with the following columns:

id Internal Word bookmark ID.

name Bookmark name.

pagenr Page number of the bookmark.

type Bookmark type inferred from the bookmark name.

Examples

```
## Not run:
bookmarkList()

## End(Not run)
```

closeWrd *Close Word Session*

Description

Closes the current Word session and optionally saves open documents.

Usage

```
closeWrd(save = FALSE)
```

Arguments

save	Logical; whether to save documents before closing.

Value

Invisibly returns `NULL`.

closeXl *Close Excel Session*

Description

Closes the current Excel session.

Usage

```
closeXl(save = FALSE)
```

Arguments

save	logical should the save dialog be displayed?

Value

Invisibly returns `NULL`.

cm_pts_conversion *Convert Between Centimeters and Points*

Description

Converts numeric values between centimeters (cm) and typographic points (pt). One centimeter corresponds to approximately 28.35 points.

Usage

```
cmToPts(x)

ptsToCm(x)
```

Arguments

x	A numeric vector.

Details

The conversion is based on:

$$1 \text{ cm} \approx 28.35 \text{ pt}$$

These functions are useful in graphical contexts (e.g., when specifying dimensions in plotting systems).

Value

A numeric vector of the same length as x, converted to the corresponding unit.

Examples

```
# Convert centimeters to points
cmToPts(1)

# Convert points to centimeters
ptsToCm(28.35)
```

getWrd *Get Current Word Session*

Description

Retrieves the current Word COM object. If no valid session exists, a new one can be created.

Usage

```
getWrd(create = TRUE)
```

Arguments

`create`	Logical; if `TRUE`, a new Word instance is created when none is available.

Value

A Word COM object or `NULL`.

`getXl` *Get Current Excel Session*

Description

Retrieves the current Excel COM object. If no valid session exists, a new one can optionally be created.

Usage

```
getXl(create = TRUE)
```

Arguments

`create`	Logical; if `TRUE`, a new Excel instance is created when none is available.

Value

An Excel COM object or `NULL`.

`newWrd` *Create a New Word Session*

Description

Starts a new Microsoft Word instance via RDCOMClient and creates an empty document. The created instance is registered as the current default Word session.

Usage

```
newWrd(visible = TRUE)
```

Arguments

`visible`	Logical; whether the Word application should be visible.

Details

The function initializes a Word application and ensures that at least one document is open. The instance is stored internally and can be accessed by other helper functions.

Value

A Word COM object of class `"COMIDispatch"`.

Examples

```
## Not run:
wrd <- newWrd()

## End(Not run)
```

---

newXl *Create a New Excel Session*

Description

Starts a new Microsoft Excel instance via RDCOMClient. The created instance is registered as the current default Excel session.

Usage

```
newXl(visible = TRUE)
```

Arguments

`visible`	Logical; whether the Excel application should be visible.

Value

An Excel COM object of class `"COMIDispatch"`.

Examples

```
## Not run:
xl <- newXl()

## End(Not run)
```

---

renameBookmark *Rename a Word Bookmark*

Description

Renames a bookmark in the active Word document while preserving its associated text range.

Usage

```
renameBookmark(name, newname, wrd = NULL)
```

Arguments

`name`	Existing bookmark name.
`newname`	New bookmark name.
`wrd`	A Word COM object. If `NULL`, the current active Word session is used.

Details

Word bookmarks cannot always be renamed reliably by directly modifying their `Name` property. Therefore, this function preserves the bookmark range, deletes the existing bookmark, and recreates it with the new name.

If a bookmark with `newname` already exists, the function returns `FALSE` and leaves the document unchanged.

Value

Invisibly returns `TRUE` on success and `FALSE` otherwise.

Examples

```
## Not run:
renameBookmark("old_name", "new_name")

## End(Not run)
```

`replaceBookmarkText` *Replace Bookmark Text*

Description

Replaces the text content of a bookmark while preserving the bookmark itself.

Usage

```
replaceBookmarkText(name, text, wrd = NULL)
```

Arguments

`name`	Bookmark name.
`text`	Replacement text.
`wrd`	A Word COM object. If `NULL`, the current active Word session is used.

Details

Word removes bookmarks automatically when their text content is replaced. This function restores the bookmark after updating the associated text range.

Value

Invisibly returns `TRUE`.

Examples

```
## Not run:
replaceBookmarkText("title", "New title")

## End(Not run)
```

`setWrd` *Set Current Word Session*

Description

Registers a Word COM object as the current default session.

Usage

```
setWrd(wrd)
```

Arguments

`wrd`	A Word COM object (class `"COMIDispatch"`).

Value

Invisibly returns `wrd`.

`setXl` *Set Current Excel Session*

Description

Registers an Excel COM object as the current default session.

Usage

```
setXl(xl)
```

Arguments

`xl`	An Excel COM object.

Value

Invisibly returns `xl`.

toWrd — *Insert Content into Microsoft Word*

Description

Inserts content into an active Microsoft Word document via RDCOMClient. The function supports different input types and dispatches methods based on the class of `x`.

Usage

```
toWrd(x, font = NULL, ..., wrd = NULL)

## Default S3 method:
toWrd(x, font = NULL, ..., wrd = NULL)

## S3 method for class 'character'
toWrd(
  x,
  font = NULL,
  para = NULL,
  style = NULL,
  bullet = FALSE,
  ...,
  wrd = NULL
)
```

Arguments

`x`	Object to be written to Word. Supported types include character vectors and arbitrary objects (converted via `capture`-like behavior).
`font`	Optional font specification (list or object of class `"font"`). If `"fix"`, a fixed-width font is used.
`...`	Additional arguments passed to methods.
`wrd`	A Word COM object. Defaults to the last active Word instance.
`para`	paragraph format
`style`	style template name
`bullet`	for bullet style

Details

The function inserts text into the current selection of a Word document. Character input is inserted directly, while other objects are first converted to text using an internal capture mechanism.

UTF-8 strings may be converted to Latin-1 depending on system locale to avoid encoding issues in Word.

Formatting options such as paragraph style, font, and bullet lists can be applied via method-specific arguments.

Value

Invisibly returns `NULL`.

Examples

```
## Not run:
toWrd("Hello World")
toWrd(1:10)
toWrd(c("Line 1", "Line 2"), bullet = TRUE)

## End(Not run)
```

---

wdConst — *Word Constants for RDCOMClient*

Description

A list of Microsoft Word constants used for automation via `RDCOMClient`. These constants correspond to enumerations defined in the Word object model and are used when calling methods on Word COM objects.

Usage

```
wdConst
```

Format

An object of class `"list"` containing named integer constants (e.g., `wdCollapseEnd`, `wdAlignParagraphLeft`, etc.).

Details

The constants are intended for use with Word COM interfaces, for example:

```
wrd[["Selection"]]$Collapse(Direction = wdConst$wdCollapseEnd)
```

This avoids the need to manually define or remember numeric values for Word enumerations.

See Also

toWrd

Examples

```
## Not run:
# Collapse selection to end of document
wrd[["Selection"]]$Collapse(Direction = wdConst$wdCollapseEnd)

# Apply paragraph alignment
wrd[["Selection"]]$ParagraphFormat()$Alignment <- wdConst$wdAlignParagraphLeft

## End(Not run)
```

withWrd — Temporarily Use a Word Session

Description

Evaluates an expression using a specified Word session as the default.

Usage

```
withWrd(wrd, expr)
```

Arguments

wrd	A Word COM object.
expr	An expression to evaluate.

Value

The result of the evaluated expression.

Examples

```
## Not run:
withWrd(newWrd(), {
  toWrd("Hello")
})

## End(Not run)
```

withXl — Temporarily Use an Excel Session

Description

Evaluates an expression using a specified Excel session as the default.

Usage

```
withXl(xl, expr)
```

Arguments

xl	An Excel COM object.
expr	An expression to evaluate.

Value

The result of the evaluated expression.

`wrdAddBookmark` *Add a Word Bookmark*

Description

Adds a new bookmark at the current cursor position in the active Word document.

Usage

```
wrdAddBookmark(name, wrd = NULL)
```

Arguments

`name`	Bookmark name.
`wrd`	A Word COM object. If `NULL`, the current active Word session is used.

Value

Invisibly returns the created bookmark COM object.

Examples

```
## Not run:
wrdAddBookmark("results")

## End(Not run)
```

`wrdBookmark` *Get a Word Bookmark*

Description

Retrieves a bookmark object from the active Word document.

Usage

```
wrdBookmark(name, wrd = NULL)
```

Arguments

`name`	Bookmark name.
`wrd`	A Word COM object. If `NULL`, the current active Word session is used.

Value

A Word bookmark COM object of class `"COMIDispatch"`, or `NULL` if the bookmark does not exist.

Examples

```
## Not run:
bm <- wrdBookmark("intro")

## End(Not run)
```

---

wrdDeleteBookmark *Delete a Word Bookmark*

Description

Deletes a bookmark from the active Word document.

Usage

```
wrdDeleteBookmark(name, wrd = NULL)
```

Arguments

`name`	Bookmark name.
`wrd`	A Word COM object. If `NULL`, the current active Word session is used.

Value

Logical. Returns `TRUE` if the bookmark was deleted, otherwise `FALSE`.

Examples

```
## Not run:
wrdDeleteBookmark("intro")

## End(Not run)
```

---

wrdGoto *Go to a Word Object*

Description

Moves the current Word selection to a specified object, such as a bookmark.

Usage

```
wrdGoto(name, what = NULL, wrd = NULL)
```

Arguments

`name`	Name of the target object.
`what`	Word GoTo constant defining the target type. Defaults to `wdConst$wdGoToBookmark`.
`wrd`	A Word COM object. If `NULL`, the current active Word session is used.

Details

The function moves the current Word selection to the specified target object using Word's `GoTo()` method.

When navigating to bookmarks, existence of the bookmark is checked before attempting the navigation.

Value

Invisibly returns `TRUE` on success and `FALSE` if the target bookmark does not exist.

Examples

```
## Not run:
wrdGoto("intro")

## End(Not run)
```

`xlDataTransferDialog` *Excel Data Transfer Dialog*

Description

Modal Tcl/Tk dialog that asks the user how to organize and deliver an Excel range that is being transferred to R. It is the interactive front-end used by `xlImport` and returns the user's choices as a plain list.

Usage

```
xlDataTransferDialog(range = NULL, multi = FALSE)
```

Arguments

`range`	character; the range address to display in the header label, e.g. `"A1:B34; C23:E55"`. `NULL` or empty shows `"(no selection)"`.
`multi`	logical; `TRUE` if several disjoint areas are selected. Restricts the structure choices and preselects `"list"`.

Details

The dialog shows the selected range address, a structure chooser, a variable name entry, and an "insert data at cursor position" checkbox.

The offered structures are:

data.frame (colnames) first row becomes the column names.

data.frame (no colnames) base-R default names `V1`, `V2`, `...`.

matrix whole range as a matrix.

table (dimnames) first column / first row as dim names.

list one component per column.

When `multi = TRUE` (a disjoint multi-area selection), only the column-combinable structures are offered - the two `data.frame` variants and `list` - and `list` is preselected, because `matrix` and `table` are not defined across several areas (see `xlParseRange`).

Value

A list with components

structure internal structure code: one of `"data.frame"`, `"data.frame.nocol"`, `"matrix"`, `"table"`, `"list"`.

variable the variable name typed by the user (possibly `""`).

insert logical; whether "insert at cursor position" was checked.

Returns `NULL` if the dialog is cancelled (Cancel button or Escape).

See Also

`xlImport`

Examples

```
## Not run:
res <- xlDataTransferDialog(range = "A1:B34")
str(res)

## End(Not run)
```

`xlGetRange` *Read the Raw Values of the Selected Excel Range(s)*

Description

Reads the values of an Excel range via RDCOMClient and returns them in a lightweight container for further processing by `xlParseRange`. No type conversion or reshaping happens here. The raw `Value2()` data is kept as-is together with the range geometry and metadata.

Usage

```
xlGetRange(xl = NULL, range = NULL)
```

Arguments

`xl`	optional Excel application handle. If `NULL`, the running Excel instance is attached via `RDCOMClient::COMCreate(..., existing = TRUE)`.
`range`	optional A1-style address (e.g. `"A1:C10"` or `"Sheet1!A1:C10"`). If `NULL`, the current selection is used.

Details

A selection may consist of several disjoint *areas* (e.g. `A1:A4` together with `C3:D5`). Each area is returned as its own `"XLRange"` block. With a single area a single block is returned; with multiple areas an (unclassed) list of blocks is returned, carrying the overall metadata as attributes.

Value

For a single area, an object of class "XLRange": a list with components values (the raw column-wise nested list from Value2()), nrow and ncol, plus attributes address, sheet and file. For multiple areas, a list of such "XLRange" objects with address, sheet and file attributes on the list itself.

Note

Value2() returns dates as numeric serial days (base 1899-12-30), not as text, and yields a column-wise nested list indexed as values[[column]][[row]].

See Also

xlParseRange, xlImport

Examples

```
## Not run:
xl <- getXl()
r  <- xlGetRange(xl)          # current selection
attr(r, "address")           # e.g. "A1:B34"
attr(r, "sheet")             # worksheet name
attr(r, "file")              # workbook (file) name

## End(Not run)
```

---

xlImport — *Interactively Import the Selected Excel Range into R*

Description

End-to-end, dialog-driven transfer of the currently selected Excel range(s) into R. The function reads the selection via xlGetRange, asks the user how to organize and deliver the data via xlDataTransferDialog, and then either returns / assigns the resulting object or inserts constructive R code at the editor cursor.

Usage

```
xlImport(xl = NULL)
```

Arguments

xl	optional Excel application handle (as returned by getXl()). If NULL, the running instance is used.

Details

The dialog offers the following target structures (single range):

data.frame (colnames) first row becomes the column names.

data.frame (no colnames) base-R default names `V1`, `V2`, `...`.

matrix whole range as a matrix, no dim names, default type choice.

table (dimnames) first column becomes row names, first row becomes column names, the top-left corner cell is discarded.

list one component per column.

When several disjoint areas are selected, only `data.frame` variants and `list` are offered (see `xlParseRange` for how areas are combined), and `list` is preselected.

Delivery is controlled by two dialog inputs, giving four combinations:

insert	variable	action
FALSE	set	`assign(variable, out, parent.frame())`
FALSE	empty	return the object (printed at the console)
TRUE	set	insert `variable` <- <code> at the cursor
TRUE	empty	insert <code> at the cursor

Value

Invisibly (or visibly, when no variable is given and nothing is inserted) the organized object: a `data.frame`, `matrix`, `list` or `table`-like matrix. Returns `invisible(NULL)` if the dialog is cancelled.

See Also

`xlGetRange`, `xlParseRange`, `xlDataTransferDialog`

Examples

```
## Not run:
xl <- getXl()
# select a range in Excel, then:
xlImport(xl)

## End(Not run)
```

`xlKill` *Terminate all Microsoft Excel processes*

Description

Forcefully terminates all running Microsoft Excel processes on Windows.

Usage

```
xlKill()
```

Details

This function calls the Windows `taskkill` command with the `/F` option. All running Excel processes are terminated immediately, and unsaved changes are lost.

Value

invisibly, the exit status returned by `taskkill`.

Note

This function is available on Windows only.

Examples

```
## Not run:
xlKill()

## End(Not run)
```

`xlParseRange` *Organize a Raw Excel Range into a data.frame, matrix, list or table*

Description

Turns the raw output of `xlGetRange` into a proper R object. Handles a single range as well as a multi-area selection, with optional header handling and automatic per-column type conversion.

Usage

```
xlParseRange(
  x,
  as = c("data.frame", "matrix", "list", "table"),
  header = FALSE,
  convert = TRUE,
  stringsAsFactors = FALSE
)
```

Arguments

- `x` an `"XLRange"` object, or a list of them (multi-area), as returned by `xlGetRange`.
- `as` target structure. One of `"data.frame"`, `"matrix"`, `"list"`, `"table"`.
- `header` logical; if `TRUE`, the first row is treated as a header (column names). Ignored for `"table"`, which always uses the first row / column as names.
- `convert` logical; if `TRUE` (default), each column is run through `.xlConvert()` for automatic numeric / Date detection.
- `stringsAsFactors` logical; passed to `data.frame` for the `"data.frame"` target. Defaults to `FALSE`.

Details

Single range. The requested structure determines the result:

`"data.frame"` with `header = TRUE` the first row supplies the column names; otherwise names are `V1, V2, ...`.

`"matrix"` the whole range as a matrix. When `convert` is `TRUE` and every column is numeric, a numeric matrix results; otherwise a character matrix.

`"list"` one component per column (named by the header when `header = TRUE`).

`"table"` first column becomes row names, first row becomes column names, and the top-left corner cell is discarded. The result is a matrix with `dimnames` (not a `table` object). Requires at least 2 rows and 2 columns.

Multiple areas. When `x` is a list of `"XLRange"` blocks (disjoint selection), only two targets are supported:

`"list"` each area is parsed as its own matrix; the result is a list of matrices.

`"data.frame"` each area is parsed as a data.frame and the areas are bound column-wise into a single data.frame. Columns of unequal length are padded with `NA` up to the longest column, and duplicate column names are disambiguated with `make.unique`.

Any other target raises an error for multi-area input.

Value

A `data.frame`, `matrix`, `list`, or a matrix with `dimnames` (for `"table"`). For a multi-area `"list"`, the returned list carries an `address` attribute.

See Also

`xlGetRange`, `xlImport`

Examples

```
## Not run:
r <- xlGetRange(xl)
xlParseRange(r, as = "data.frame", header = TRUE)
xlParseRange(r, as = "matrix")
xlParseRange(r, as = "table")     # first col = rownames, first row = colnames

## End(Not run)
```

`xlView` *View a Data Frame in Excel*

Description

Opens a data frame in Microsoft Excel for interactive inspection.

Usage

```
xlView(
  x,
  sheet = deparse(substitute(x)),
  rowNames = FALSE,
  table = TRUE,
  autofit = TRUE,
  freeze = TRUE,
  xl = NULL
)
```

Arguments

`x`	A data frame.
`sheet`	Optional worksheet name.
`rowNames`	Logical; if `TRUE`, row names are included.
`table`	Logical; if `TRUE`, Excel filters are enabled.
`autofit`	Logical; if `TRUE`, column widths are adjusted automatically.
`freeze`	Logical; if `TRUE`, the first row is frozen.
`xl`	Optional Excel COM object.

Details

A new workbook and worksheet are created automatically.

Data are transferred column-wise to preserve numeric types. Factors and date-time objects are converted to character vectors.

The workbook is not saved automatically.

Value

Invisibly returns the worksheet COM object.

Examples

```
## Not run:
xlView(iris)
xlView(mtcars)

## End(Not run)
```

`xxlView` *Run xlView() on selected text.*

Description

Run xlView() on selected text.

Usage

```
xxlView()
```

Index